

Числа с плавающей точкой

Повторение

- ▶ Как в компьютере представлены целые числа?
- ▶ Какие проблемы будут, если так же представить дробные числа?

Повторение

- ▶ Как в компьютере представлены целые числа?
- ▶ Какие проблемы будут, если так же представить дробные числа?
- ▶ Какие решения этих проблем вы знаете?

Зачем нужен стандарт

Если бы каждый процессор использовал свою систему дробных чисел:

- ▶ программы стали бы непереносимыми;
- ▶ одна и та же программа давала бы разные результаты (разные ошибки округления).

Поэтому введён единый стандарт — **IEEE 754**, общий для всех компьютеров и архитектур.

Диапазон чисел

Хотим хранить и очень маленькие, и очень большие числа.

Позиционная запись

n бит: $\sim 2^n$

каждый бит удваивает
диапазон

Экспоненциальная запись

w бит порядка: $\sim 2^{2^w-1}$

каждый бит возводит диапазон
в квадрат

float32: от $\approx 10^{-45}$ до $\approx 3.4 \cdot 10^{38}$ за 32 бита. В позиционной записи для этого нужно ≈ 277 бит.

Формат IEEE 754

Число хранится как тройка полей (S , e , m):

- ▶ S — знак
- ▶ e — порядок (со смещением)
- ▶ m — дробная часть мантииссы

Формат	S	e	m
float32	1 бит	8 бит	23 бита
float64	1 бит	11 бит	52 бита
учебный	1 бит	3 бита	4 бита

Нормализованная форма

$$x = (-1)^S \cdot (1.m)_2 \cdot 2^E, \quad E = e - \text{bias}$$

- ▶ $\text{bias} = 2^{k-1} - 1$, где k — число бит порядка: 127 для float32, 3 для учебного формата
- ▶ e хранится без знака, со смещением
- ▶ ведущая единица не хранится
- ▶ нормализованные числа: $1 \leq e \leq 2^k - 2$

Очень маленькие числа

Когда невыгодно хранить число в нормализованной форме?

Очень маленькие числа

Когда невыгодно хранить число в нормализованной форме?

- ▶ Когда число близко к нулю

Очень маленькие числа

Когда невыгодно хранить число в нормализованной форме?

- ▶ Когда число близко к нулю
- ▶ Наименьшее нормализованное число float32:
 $1.0 \cdot 2^{-126} \approx 1.18 \cdot 10^{-38} \text{ (} e = 1 \text{)}$

Денормализованная форма

Числа с $e = 0$ и $m \neq 0$:

$$x = (-1)^S \cdot (0.m)_2 \cdot 2^{-126}$$

- ▶ целая часть мантиисы — 0 (скрытый бит равен 0)
- ▶ порядок фиксирован: $E = -126$, как у $e = 1$ (а не -127) — так нет разрыва с нормализованными
- ▶ чем ближе к 0, тем меньше значащих бит
- ▶ у каждого числа одна кодировка: $e \geq 1$ — скрытый бит 1, $e = 0$ — скрытый бит 0

Пример: $1/8$ в 8 битах

Учебный формат: $\text{bias} = 3$.

1. $1/8 = 0.001_2 = 1.0000_2 \cdot 2^{-3}$, значит $E = -3$
2. $e = E + 3 = 0$ — но $e = 0$ означает денормализованное число
3. записываем как $0.m \cdot 2^{-2}$: $1/8 = 0.1000_2 \cdot 2^{-2}$
4. $S = 0$, $e = 000$, $m = 1000$: 0_000_1000

Пример: 8-битные числа

число	S_e_m	вид
1/4	0_001_0000	наименьшее нормализованное
15/64	0_000_1111	наибольшее денормализованное
1/8	0_000_1000	денормализованное
1/64	0_000_0001	наименьшее положительное
0	0_000_0000	ноль
15.5	0_110_1111	наибольшее конечное

От 0 до 1/2 числа идут с постоянным шагом 1/64.

Особые значения

	S	e	m
$+\infty$	0	1...1	0...0
$-\infty$	1	1...1	0...0
NaN	любой	1...1	$\neq$ 0...0
± 0	0 / 1	0...0	0...0

В 8 битах: $+\infty - 0_111_0000$, $-\infty - 1_111_0000$.

Особые значения в C

```
float pos_inf = 1.0f / 0.0f;    // +inf  
float neg_inf = -1.0f / 0.0f;   // -inf  
float nan_val = 0.0f / 0.0f;    // NaN
```

- ▶ NaN != NaN
- ▶ +0 и -0 — разные битовые представления (но +0 == -0)
- ▶ деление на 0 не завершает программу, а выставляет флаги

Перевод в IEEE 754

Учебный формат: 1 бит знака, 3 бита порядка, 4 бита мантииссы;
 $\text{bias} = 3$.

1. знак: $S = 0$ при $x \geq 0$, иначе $S = 1$; дальше берём $|x|$
2. перевести $|x|$ в двоичную систему
3. нормализовать: $|x| = 1.\text{xxxx} \dots_2 \cdot 2^E$
4. порядок: $e = E + 3$ (если $e \leq 0$ — денормализованное число)
5. m — первые 4 бита после запятой (с округлением)

Пример: 5.75

1. $x > 0$, значит $S = 0$
2. $5.75 = 101.11_2$
3. $101.11_2 = 1.0111_2 \cdot 2^2$, значит $E = 2$
4. $e = E + 3 = 5 = 101_2$
5. $m = 0111$

Ответ: 0_101_0111

Упражнение

Переведите числа в 8-битный формат:

$$1\frac{1}{4}$$

$$-\frac{3}{8}$$

$$-\frac{2}{3}$$

$$-\infty$$

Упражнение

Переведите числа в 8-битный формат:

$$1\frac{1}{4} \quad 0_011_0100$$

$$-\frac{3}{8} \quad 1_001_1000$$

$$-\frac{2}{3} \quad 1_010_0101$$

$$-\infty \quad 1_111_0000$$

Ошибка округления

Разбор третьего числа: $-\frac{2}{3}$.

► $\frac{2}{3} = 0.\overline{10}_2 = 1.0101\dots_2 \cdot 2^{-1}$

► $e = -1 + 3 = 2 = 010$

► в мантиссу помещаются 4 бита: $m = 0101$, остальное отбрасывается

► получили $1_010_0101 = -\frac{21}{32} = -0.65625 \neq -\frac{2}{3}$

Точно представить число нельзя — результат округляется до ближайшего представимого.

Почему 0.1 непредставимо

$$0.1 = 0.0001\ 1001\ 1001\ 1001\ \dots_2 = 1.\overline{1001}_2 \cdot 2^{-4}$$

- ▶ бесконечная периодическая дробь — как $\frac{1}{3} = 0.333\dots$ в десятичной записи
- ▶ в конечное число бит она не помещается, значит, округляется
- ▶ в double: $0.1 \approx 0.100000000000000000055\dots$

$$0.1 + 0.2 \neq 0.3$$

```
double a = 0.1 + 0.2;
printf("%d\n", a == 0.3);    // 0
printf("%.17g\n", a);       // 0.3000000000000000004
printf("%.17g\n", 0.3);     // 0.2999999999999999999
```

Ни 0.1, ни 0.2, ни 0.3 не представимы точно, ошибки округления накапливаются.

Сравнение чисел

- ▶ $a == b$ для fp-чисел почти всегда неверно
- ▶ сравниваем только через порог: $|a - b| < \varepsilon$

`fabs(a - b) < eps`

Пример: $|(0.1 + 0.2) - 0.3| \approx 5.55 \cdot 10^{-17}$

Системный eps

Системный (машинный) ε — расстояние от 1.0 до следующего представимого числа: $\varepsilon = 2^{-t}$, где t — число бит мантииссы.

формат	ε	в C
float32	$2^{-23} \approx 1.19 \cdot 10^{-7}$	FLT_EPSILON
float64	$2^{-52} \approx 2.22 \cdot 10^{-16}$	DBL_EPSILON
учебный	$2^{-4} = 0.0625$	

Здесь $5.55 \cdot 10^{-17} < \varepsilon$, поэтому $0.1 + 0.2 \approx 0.3$.

Системный eps: масштаб

ε — точность около 1.0. Для чисел другого порядка шаг между соседними числами другой.

Для больших и малых чисел сравнивайте относительно:

$$|a - b| < \varepsilon \cdot \max(|a|, |b|)$$

Схема операции

IEEE 754 стандартизирует не только хранение чисел, но и работу с ними:

1. распаковка: извлекаются S , E , M
2. проверка особых значений: NaN, $\pm\infty$
3. выполнение арифметики
4. нормализация: результат приводится к $1.xxxx \cdot 2^E$
5. округление
6. упаковка: S , E , M собираются обратно

Сложение и вычитание

1. число с меньшим порядком сдвигаем вправо, пока $E_1 = E_2$
2. мантиссы складываются (вычитаются)
3. знак результата берётся по результату вычисления
4. нормализация и округление

Пример: $A + B$

$$A = \frac{5}{4} = 0_011_0100, \quad B = \frac{3}{8} = 0_001_1000$$

$$A = 1.0100 \cdot 2^0, \quad B = 1.1000 \cdot 2^{-2}$$

- ▶ выравниваем порядки: $B = 0.0110 \cdot 2^0$ (сдвиг вправо на 2)
- ▶ складываем мантиссы: $1.0100 + 0.0110 = 1.1010$
- ▶ ответ: $1.1010 \cdot 2^0 = \frac{13}{8} = 0_011_1010$

Пример: $A - B$

$$A = \frac{5}{4} = 0_011_0100, \quad B = \frac{3}{8} = 0_001_1000$$

$$A = 1.0100 \cdot 2^0, \quad B = 1.1000 \cdot 2^{-2}$$

- ▶ выравниваем порядки: $B = 0.0110 \cdot 2^0$
- ▶ вычитаем мантиссы: $1.0100 - 0.0110 = 0.1110$
- ▶ нормализуем: $0.1110 \cdot 2^0 = 1.1100 \cdot 2^{-1}$, $e = -1 + 3 = 2$
- ▶ ответ: $\frac{7}{8} = 0_010_1100$

Умножение и деление

Умножение

- ▶ $S = S_1 \oplus S_2$
- ▶ $E = E_1 + E_2$
($e = e_1 + e_2 - 3$)
- ▶ $M = M_1 \cdot M_2 \in [1, 4)$
- ▶ если $M \geq 2$: $M/2, E + 1$

Деление

- ▶ $S = S_1 \oplus S_2$
- ▶ $E = E_1 - E_2$
($e = e_1 - e_2 + 3$)
- ▶ $M = M_1 / M_2 \in (0.5, 2)$
- ▶ если $M < 1$: $2M, E - 1$

Затем округление.

Пример: $A \cdot B$

$$A = \frac{5}{4} = 0_011_0100, \quad B = -\frac{3}{8} = 1_001_1000$$

$$\blacktriangleright S = 0 \oplus 1 = 1$$

$$\blacktriangleright E = 0 + (-2) = -2, \quad e = -2 + 3 = 1 = 001$$

$$\blacktriangleright M = 1.0100_2 \cdot 1.1000_2 = 1.1110_2$$

($1.25 \cdot 1.5 = 1.875 < 2$, нормализация не нужна)

$$\text{Ответ: } 1_001_1110 = -\frac{15}{32}$$

Упражнение: A/B

$$A = \frac{5}{4} = 0_011_0100, \quad B = -\frac{3}{8} = 1_001_1000$$

$$A/B = ?$$

Упражнение: A/B

$$A = \frac{5}{4} = 0_011_0100, \quad B = -\frac{3}{8} = 1_001_1000$$

$$A/B = ?$$

$$\blacktriangleright S = 1, \quad E = 0 - (-2) = 2$$

$$\blacktriangleright M = 1.25/1.5 = 0.8333\dots < 1, \text{ значит } M = 1.6666\dots, \quad E = 1$$

Упражнение: A/B

$$A = \frac{5}{4} = 0_011_0100, \quad B = -\frac{3}{8} = 1_001_1000$$

$$A/B = ?$$

- ▶ $S = 1, \quad E = 0 - (-2) = 2$
- ▶ $M = 1.25/1.5 = 0.8333... < 1$, значит $M = 1.6666..., \quad E = 1$
- ▶ округляем: $1.1010\ 1010..._2 \rightarrow 1.1011_2, \quad e = 1 + 3 = 100$
- ▶ ответ: $1_100_1011 = -3.375$ (точно: $-\frac{10}{3}$)

Что должен знать программист

1. NaN != NaN
2. +0 и -0 — разные фп-числа
3. деление на 0 для фп не завершает программу, а выставляет флаги
4. фп нельзя сравнивать напрямую, только через ε
5. перед суммированием массива фп его лучше отсортировать (по возрастанию модуля)
6. сложение фп неассоциативно: $a + (b + c) \neq (a + b) + c$